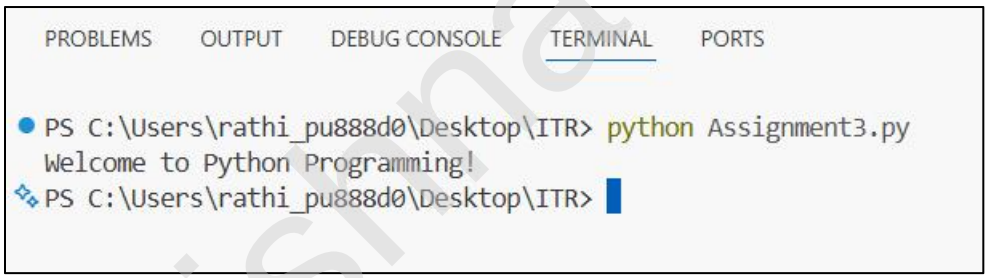
# Session 03 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**
College: **Zeal Polytechnic, Narhe, Pune**
Branch: **Artificial Intelligence and Machine Learning (AIML)**
Domain: **AIML (Artificial Intelligence and Machine Learning)**

Q1. Basic Functions

**Output Screenshot:**

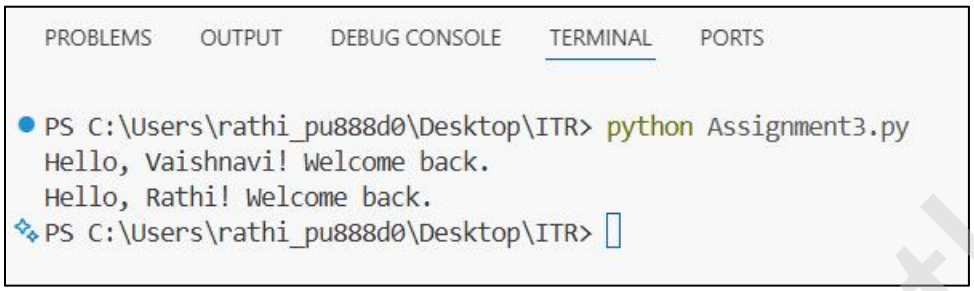


**Program Code:**

```
""" ASSIGNMENT 3 """

'''
Q1. (Basic Function)

Write a function named welcome() that prints the message:
"Welcome to Python Programming!"

Call the function to execute it.

Add a comment explaining why functions are useful.
'''

# Functions help organize code, avoid repetition, and make programs easier to understand.

def welcome():
    print("Welcome to Python Programming!")

# Calling the function
welcome()
```

Q2. Function with Parameter

**Output Screenshot:**

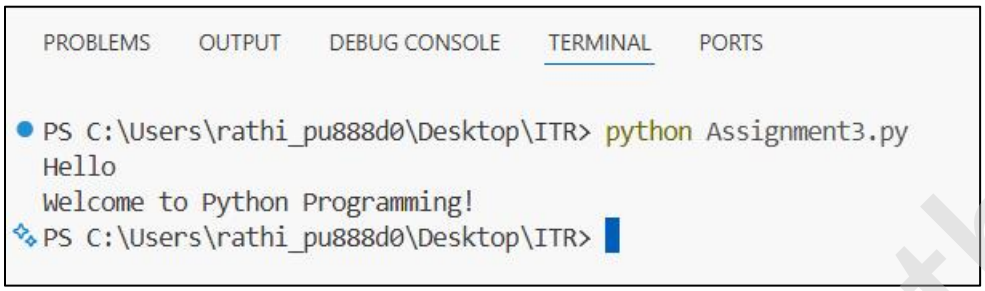



**Program Code:**

```
""" ASSIGNMENT 3 """

'''
Q2. (Function with Parameter)

Create a function named greet(name) that takes one parameter name and prints:
"Hello, [name]! Welcome back."

Call the function with your own name.
Also call it with a different name to show reusability.
'''

def greet(name):
    print("Hello,", name + "! Welcome back.")

# Calling the function with my own name
greet("Vaishnavi")

# Calling the function with a different name
greet("Rathi")

'''Explanation in Comments'''
# The parameter 'name' receives the value passed to the function.
# The function prints a personalized greeting message.
# The same function can be used with different names, showing reusability.
```

Q3. Default Parameter

**Output Screenshot:**

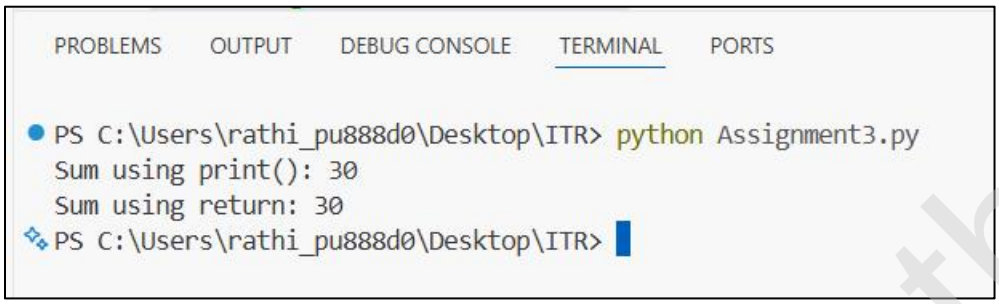



**Program Code:**

```
""" ASSIGNMENT 3 """

'''
Q3. (Default Parameter)
Write a function show_message(message="Hello") that prints the message.
Call the function without passing any argument.
Call the function again by passing a different message.
Explain the benefit of default parameters in a comment.
'''

# Default parameters allow a function to use a predefined value when no argument is provided.

def show_message(message="Hello"):
    print(message)

# Calling the function without passing an argument
show_message()

# Calling the function with a different message
show_message("Welcome to Python Programming!")

'''Explanation in Comments'''
# The parameter 'message' has a default value of "Hello".
# If no argument is passed, the default value is used.
# If an argument is provided, it replaces the default value.
# Default parameters make functions more flexible and easier to use.
```

Q4. Function with Multiple Parmeters & Return

**Output Screenshot:**




**Program Code:**

```
""" ASSIGNMENT 3 """

'''
Q4. (Function with Multiple Parameters & Return)
Create a function calculate_sum(a, b) that takes two numbers and returns their sum.
Call the function with two numbers and store the result in a variable.
Print the result.
Also demonstrate the difference between print() inside the function and using return.
'''

# Function using print()
def show_sum(a, b):
    print("Sum using print():", a + b)

# Function using return
def calculate_sum(a, b):
    return a + b

# Calling the function that uses print()
show_sum(10, 20)

# Calling the function that uses return
result = calculate_sum(10, 20)

# Printing the returned result
print("Sum using return:", result)
```

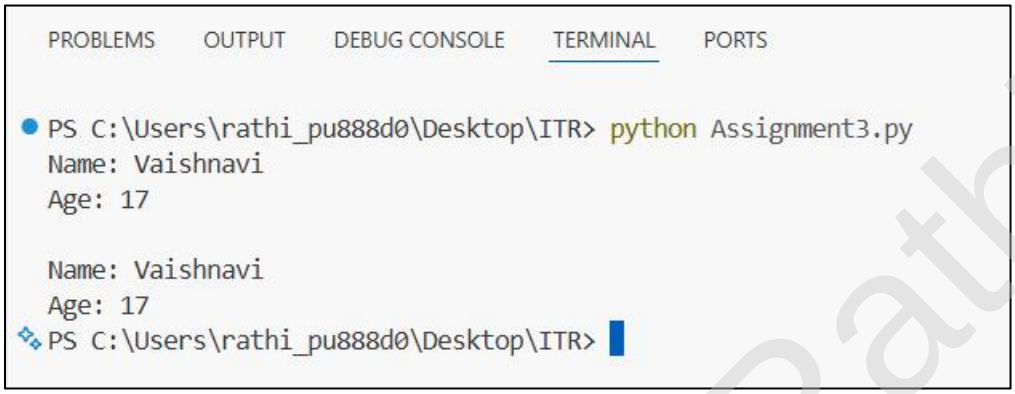
'''Explanation in Comments'''

# The function show_sum() directly displays the result using print().
# The function calculate_sum() returns the result to the calling code.
# The returned value is stored in the variable 'result'.
# A value returned by a function can be reused later in the program.
# print() only displays output, whereas return sends the value back to the caller.

---

Q5. Positional vs Keyword Argument

**Output Screenshot:**




**Program Code:**

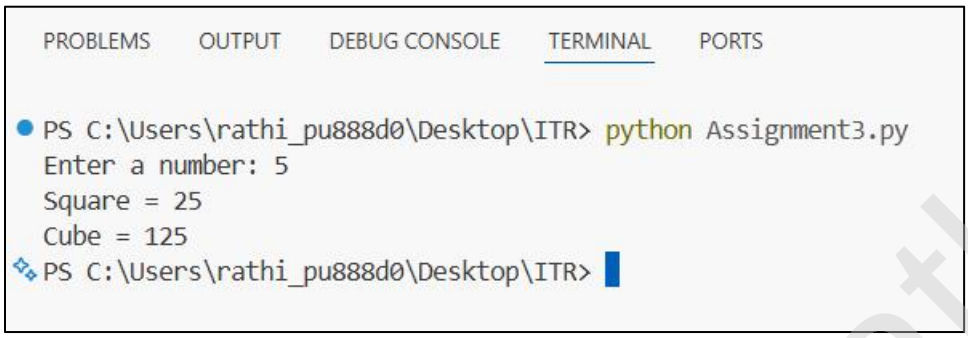
```
""" ASSIGNMENT 3 """

'''
Q5. (Positional vs Keyword Arguments)

Write a function student_info(name, age) that prints the name and age.
Call this function twice:
1. Using positional arguments.
2. Using keyword arguments.
Add comments explaining the difference.
'''

def student_info(name, age):
    print("Name:", name)
    print("Age:", age)

# Calling the function using positional arguments
student_info("Vaishnavi", 17)

print()

# Calling the function using keyword arguments
student_info(age=17, name="Vaishnavi")
```

'''Explanation in Comments'''
# In positional arguments, values are passed according to their position.
# The first value is assigned to 'name' and the second value to 'age'.
# In keyword arguments, parameter names are specified while calling the function.
# The order of arguments does not matter when using keyword arguments.
# Both methods pass values to the function, but keyword arguments improve readability.

---

Q6. Function with Return value

**Output Screenshot:**




**Program Code:**

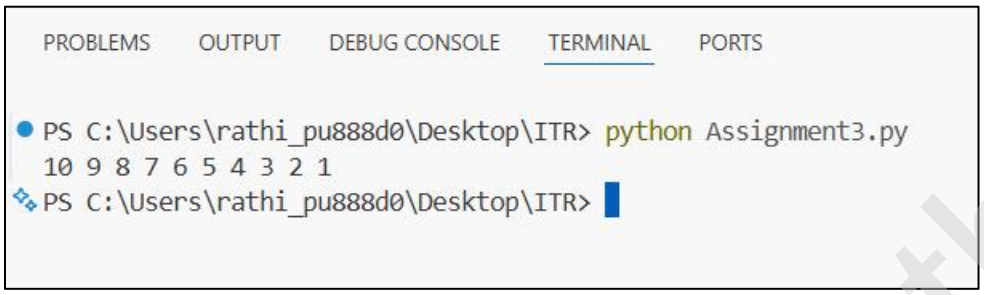
```
""" ASSIGNMENT 3 """

'''
Q6. (Function with Return Value)

Write a function square(num) that returns the square of the given number.
Write another function cube(num) that returns the cube of the number.
In the main program, take a number as input from the user and print its square and
cube using these functions.
'''

# Function to return the square of a number
def square(num):
    return num ** 2

# Function to return the cube of a number
def cube(num):
    return num ** 3

# Take input from the user
number = int(input("Enter a number: "))

# Call the functions and store the results
square_result = square(number)
cube_result = cube(number)

# Display the results
print("Square =", square_result)
print("Cube =", cube_result)
```

'''Explanation in Comments'''
# The function square() returns the square of the given number.
# The function cube() returns the cube of the given number.
# The user enters a number in the main program.
# The returned values are stored in variables and then printed.
# Using separate functions makes the program modular and reusable.

---

Q7. Recursion - Basic

**Output Screenshot:**




**Program Code:**

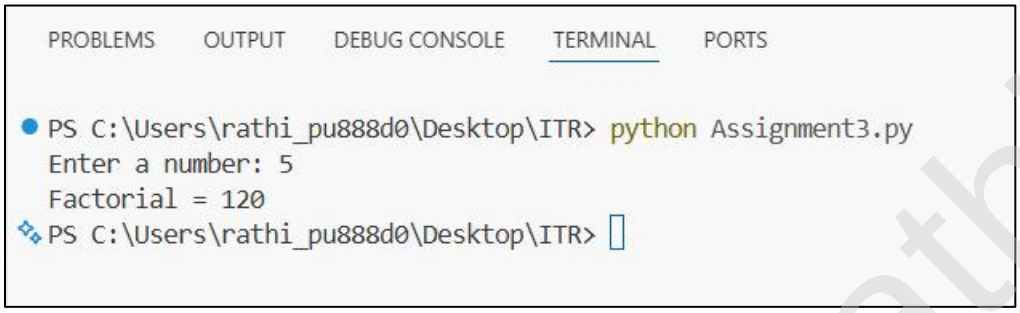
```
""" ASSIGNMENT 3 """

'''
Q7. (Recursion - Basic)

Write a recursive function countdown(n) that prints numbers from n down to 1.
Call countdown(10).
Make sure to include the base case (stopping condition).
'''

# Recursive function to print numbers from n down to 1
def countdown(n):

    # Base case: stop recursion when n becomes 0
    if n == 0:
        return

    print(n, end=' ')  # Print the current number followed by a space

    # Recursive call with n decreased by 1
    countdown(n - 1)

# Call the function
countdown(10)
```

```
'''Explanation in Comments'''
# Recursion is a technique where a function calls itself.
# The base case prevents the function from calling itself forever.
# When n becomes 0, the function stops.
# Each recursive call decreases n by 1.
# The function prints numbers from 10 down to 1.
```

---

Q8. Recursion - Factorial

**Output Screenshot:**




**Program Code:**

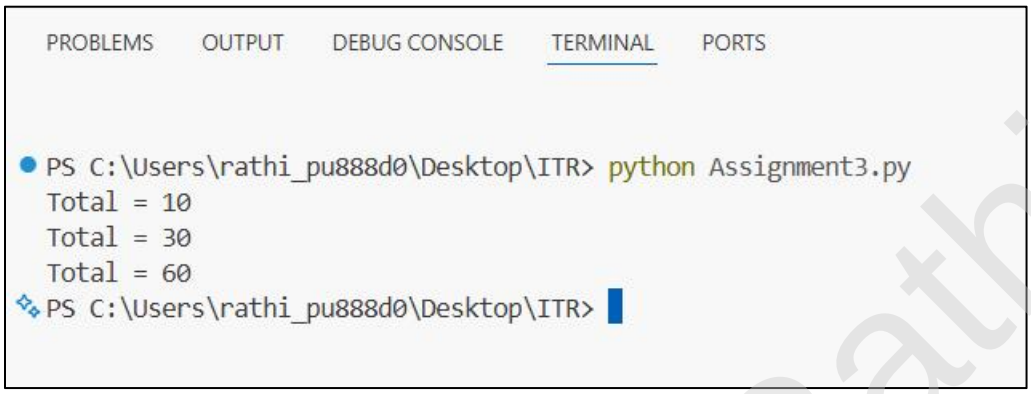
```
""" ASSIGNMENT 3 """

'''
Q8. (Recursion - Factorial)
Write a recursive function factorial(n) that calculates and returns the factorial of a
number.
Example: factorial(5) should return 120.
Test it with input from the user and print the result.
Add a comment explaining the base case.
'''

# Recursive function to calculate factorial
def factorial(n):

    # Base case: factorial of 0 or 1 is 1, so recursion stops here
    if n == 0 or n == 1:
        return 1

    return n * factorial(n - 1)

# Take input from the user
number = int(input("Enter a number: "))

# Print the factorial
print("Factorial =", factorial(number))
```

```
'''Explanation in Comments'''
# Recursion is a process where a function calls itself.
# The base case is used to stop the recursive calls.
# factorial(n) = n * factorial(n - 1)
# The function keeps calling itself until n becomes 0 or 1.
# Then the results are multiplied and returned back.
```

---

Q9.Scope - Local vs Global

**Output Screenshot:**




**Program Code:**

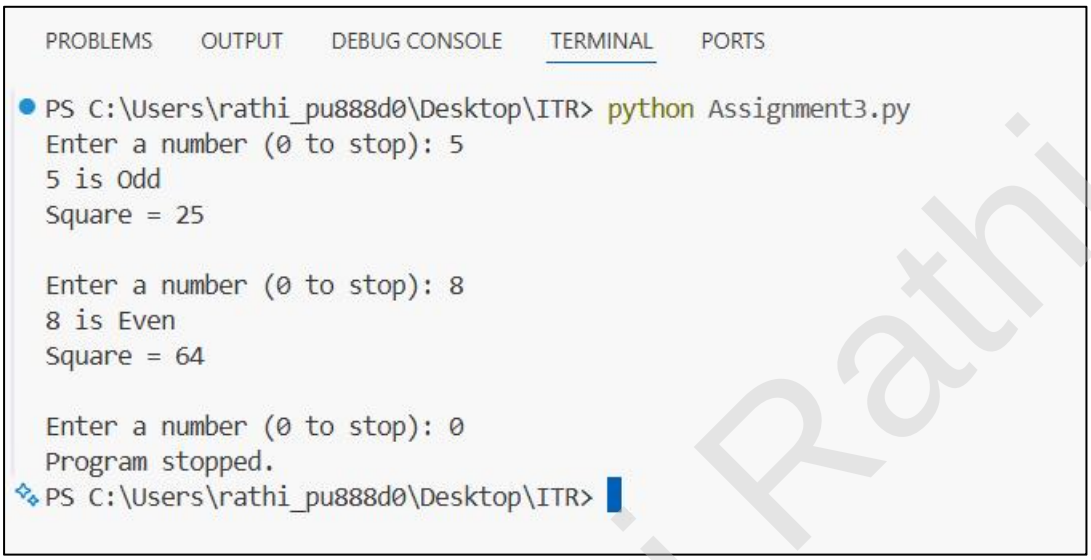
```
""" ASSIGNMENT 3 """

'''
Q9. (Scope - Local vs Global)
Write a program to demonstrate scope:
Create a global variable total = 0
Write a function add_value(x) that adds x to total (use the global keyword).
Call the function multiple times and print the value of total after each call.
Also try to print a local variable outside its function to show it doesn't work.
'''

# Global variable
total = 0

def add_value(x):
    global total

    # Local variable
    local_var = x

    # Add x to the global variable total
    total += x

    print("Total =", total)
```

```
# Calling the function multiple times
add_value(10)
add_value(20)
add_value(30)

# Trying to access a local variable outside the function
# This will cause an error because local_var exists only inside the function
# print(local_var)

'''Explanation in Comments'''
# 'total' is a global variable and can be accessed throughout the program.
# The global keyword allows the function to modify the global variable.
# 'local_var' is a local variable and exists only inside add_value().
# Trying to access local_var outside the function results in a NameError.
# This demonstrates the difference between global and local scope.
```

---

Q10. Mini Project - All Concepts

**Output Screenshot:**




**Program Code:**

```
""" ASSIGNMENT 3 """

'''
Q10. (Mini Project - All Concepts)

Create a Simple Number Analyzer program using functions:
1. get_number() → takes input from user and returns the number.
2. is_even(num) → returns True if number is even, else False.
3. power(base, exp=2) → returns base raised to exponent (use default argument).
4. show_result(num) → prints whether the number is even/odd and its square.

Use a while loop to repeatedly ask the user for numbers until they enter 0.
Use recursion only if needed for one small part.
'''

# Function to take input from the user
def get_number():
    return int(input("Enter a number (0 to stop): "))

# Function to check whether a number is even
def is_even(num):
    return num % 2 == 0
```

```
# Function with a default argument
def power(base, exp=2):
    return base ** exp

# Function to display the result
def show_result(num):

    if is_even(num):
        print(num, "is Even")
    else:
        print(num, "is Odd")

    print("Square =", power(num))

# Main program using a while loop
while True:

    number = get_number()

    # Stop the program when the user enters 0
    if number == 0:
        print("Program stopped.")
        break

    show_result(number)
    print()

'''Explanation in Comments'''
# get_number() takes input from the user and returns it.
# is_even() checks whether the number is even or odd.
# power() calculates the power of a number and uses a default exponent of 2.
# show_result() displays whether the number is even or odd and prints its square.
# The while loop keeps asking for numbers until the user enters 0.
# Recursion is not required for this program because a while loop is sufficient.
```

---